

SAMPLE WEB APPLICATION & API PENETRATION TEST REPORT

Northstar SaaS Demo

IMPORTANT NOTICE Sanitized fictional sample - not a client engagement. No real system was tested, and no real client or employer information is included.

Prepared by	Vinit Vora
Role	Application Security Engineer
Services	Web & API Penetration Testing AWS & Kubernetes Security
Report version	1.0
Assessment date	10-12 August 2026 (fictional)
Classification	Public demonstration sample

contact@vinitvora.com | <https://vinitvora.com>

GOVERNANCE

Document control

Document title	Sample Web Application & API Penetration Test Report
Version	1.0
Status	Final sample for review
Prepared by	Vinit Vora
Intended audience	Prospective clients, engineering leaders, product owners and security stakeholders
Classification	Public demonstration sample
Distribution	May be shared publicly only as a fictional sample; not evidence of a real engagement

Revision history

Version	Date	Author	Change
1.0	14 September 2026	Vinit Vora	Initial fictional demonstration report

Table of contents

#	Section	Page
1	Important notice and limitations	3
2	Executive summary	4
3	Engagement overview, scope and rules	5
4	Methodology and severity	6
5	Findings summary	7
6	NST-001 - Broken object-level authorization	8
7	NST-002 - Unsafe mass assignment	10
8	NST-003 - Insufficient rate limiting	11
9	NST-004 - Cookie-security attributes	12
10	NST-005 - Positive security observations	13
11	Remediation roadmap and retest process	14
12	Conclusion and contact	15

SCOPE ASSURANCE

1. Important notice and limitations

This document is a completely fictional, sanitized demonstration of report structure, writing quality and remediation detail. Northstar SaaS Demo does not represent a real client, employer, freelance engagement, production system or known product. No real system was tested to create this report.

AUTHORIZATION REQUIRED Real penetration testing requires explicit written authorization, a mutually agreed scope, approved testing conditions and clear points of contact before any testing begins.

Assessment limitations

- Penetration testing provides a point-in-time view of observable security conditions during the agreed testing window.
- No assessment can guarantee discovery of every vulnerability, attack path, configuration weakness or future defect.
- Coverage depends on the supplied scope, access level, test data, architecture visibility, time available and operational constraints.
- Changes made after testing may introduce, remove or alter security risks.
- Automated tooling is used selectively; findings require manual validation before they are reported as confirmed issues.
- Retesting validates the supplied remediation for agreed findings and is not automatically equivalent to a complete second penetration test.

Interpretation

Severity ratings in this sample reflect a risk-based professional judgment using exploitability, required access, data sensitivity, business impact, affected population, integrity and availability consequences, and existing mitigating controls. They are not contractual guarantees or formal certification statements.

STAKEHOLDER SUMMARY

2. Executive summary

Northstar SaaS Demo is a fictional multi-tenant software-as-a-service application with authenticated web and API (application programming interface) workflows. The fictional objective was to assess whether one authenticated user could cross account or object boundaries, whether sensitive profile attributes were safely controlled, and whether selected abuse-resistant controls behaved as expected.

Overall posture

The sample assessment identified one High, two Medium and one Low severity security finding, together with several positive observations. The most important issue was a broken object-level authorization condition in which the server trusted a user-supplied account reference without consistently confirming ownership. This type of weakness can undermine otherwise sound authentication because a valid session alone does not prove that the requester may access every referenced object.

Principal risk themes

Theme	What it means for stakeholders	Priority
Authorization boundaries	Authenticated users must be checked against the specific resource and permitted action on every request.	Immediate
Unsafe field binding	Client-supplied fields should not map directly to sensitive server-side model attributes.	Short term
Abuse resistance	Sensitive workflows need deliberate limits and detection logic, not only generic performance controls.	Short term
Session hardening	Cookie attributes should reduce avoidable browser-side exposure and cross-site risk.	Routine hardening

Recommended action

Correct object-level authorization first, then review related resource families for the same authorization pattern. Introduce explicit field allowlists for user-controlled updates, strengthen abuse controls around sensitive actions, and standardize session-cookie attributes. A focused retest should confirm that fixes are effective and do not leave equivalent bypasses in adjacent endpoints.

ENGAGEMENT DEFINITION

3. Engagement overview, scope and rules of engagement

Engagement overview

Assessment type	Authenticated web application and API (application programming interface) penetration test
Assessment dates	10-12 August 2026 (fictional)
Approach	Manual-first, risk-based testing with targeted automated coverage
Environment	Dedicated fictional test environment
Access provided	Two controlled standard-user accounts and one limited administrative test role
Deliverables	Technical report, executive summary, remediation guidance and one focused retest
Retest terms	Verification of reported findings within the agreed retest window
Communication	Daily status note; immediate escalation for material service risk or unintended impact

In scope

- app.example.test
- api.example.test
- Authenticated user workflows
- Account and object-ownership boundaries
- Selected administrative functionality
- Session and authentication handling

Explicit exclusions

Denial-of-service testing; social engineering; physical security; mobile applications; cloud infrastructure; third-party services; source-code review; destructive testing; and any asset not expressly listed above.

Rules of engagement

- Written authorization and an approved testing window are mandatory.
- Only controlled test accounts and synthetic data are used.
- Evidence collection follows the minimum-necessary principle; no bulk extraction is permitted.
- An emergency-stop contact and procedure are agreed before testing begins.
- No persistence mechanisms are created.
- Reports and evidence are transferred securely and retained only for the agreed period, then deleted according to the engagement terms.

TESTING APPROACH

4. Methodology and severity methodology

The assessment methodology is informed by the OWASP (Open Worldwide Application Security Project) Web Security Testing Guide, OWASP Application Security Verification Standard and OWASP API Security Top 10. These references guide coverage and terminology; this report does not imply OWASP endorsement, accreditation or certification.

Practical workflow

1. Confirm authorization, scope, exclusions and test identities.
2. Map authenticated application and API workflows, roles, object references and trust boundaries.
3. Manually test authentication, session handling, object- and function-level authorization, input handling and business logic.
4. Use targeted automated checks to broaden coverage where they add value without creating avoidable operational risk.
5. Manually confirm suspected weaknesses and gather minimum necessary evidence.
6. Assess business impact and technical risk in context, then provide developer-ready remediation guidance.
7. Verify agreed remediation during a focused retest.

Severity matrix

Severity	Typical interpretation	Expected response
Critical	Direct, reliable path to severe compromise with broad impact and limited prerequisites.	Emergency remediation and executive escalation.
High	Material confidentiality, integrity or privilege impact with practical exploitation conditions.	Prioritize and remediate quickly.
Medium	Meaningful security weakness requiring specific access, conditions or chaining.	Plan remediation in the near term.
Low	Limited impact or hardening gap with constrained exploitability.	Address through routine hardening.
Informational	Positive control, observation or improvement note without a confirmed exploitable weakness.	Track as appropriate.

Severity considers exploitability, required access, data sensitivity, business impact, scope of affected users, integrity and availability consequences, and existing mitigating controls. CVSS (Common Vulnerability Scoring System) scores are intentionally omitted from this sample to avoid implying unsupported environmental precision.

RESULTS

5. Findings summary

SAMPLE ONLY All findings below are fictional and exist solely to demonstrate how confirmed issues would be documented after an authorized assessment.

ID	Finding	Severity	Affected component	Status
NST-001	Broken object-level authorization	High	api.example.test / account object access	Open
NST-002	Unsafe mass assignment in profile update	Medium	api.example.test / profile update	Open
NST-003	Insufficient rate limiting on sensitive workflow	Medium	api.example.test / recovery verification	Open
NST-004	Incomplete session-cookie security attributes	Low	app.example.test / session cookie	Open
NST-005	Positive security observations	Informational	Selected application controls	Observed

Risk concentration

The dominant risk theme is server-side authorization. Authentication establishes who the requester is; authorization must still determine whether that identity may act on the specific account, object and operation. The remaining findings show common secondary control gaps around server-side field enforcement, workflow abuse resistance and session hardening.

Prioritization logic

- Fix NST-001 first and search for the same authorization pattern across adjacent endpoints and resource families.
- Address NST-002 and NST-003 next because both can turn otherwise legitimate functionality into unintended privilege or abuse paths.
- Standardize NST-004 through application and platform configuration to prevent regressions across environments.

DETAILED TECHNICAL FINDING

6. NST-001 - Broken object-level authorization

Severity	High
Status	Open
Affected component	api.example.test - account object access
CWE	CWE-639: Authorization Bypass Through User-Controlled Key
OWASP mapping	OWASP API Security Top 10 - API1: Broken Object Level Authorization

Description

The account-detail workflow accepted a client-supplied account identifier and returned the referenced object after validating only that the requester had a valid authenticated session. The server did not consistently enforce that the authenticated user owned, or was otherwise authorized to access, the requested account object.

Preconditions

- A valid authenticated session for a controlled standard-user account.
- Knowledge or discovery of another valid fictional account reference within the test environment.
- Access to the in-scope account-detail API workflow.

Business impact

If present in a real multi-tenant service, this weakness could allow one customer to access another customer's account-level information and could undermine tenant separation. Impact would depend on the sensitivity of the returned object and whether equivalent authorization gaps existed on update or delete actions.

Technical risk

The application treats possession of an object reference as sufficient authorization. Because authorization is not bound to the authenticated identity and requested action, modifying the reference can cross an ownership boundary even though the user remains correctly authenticated.

Sanitized validation procedure

8. Authenticate as user_test_a and confirm that the account object acct_demo_001 is legitimately accessible.
9. Submit the same read operation using a different controlled account reference belonging to user_test_b.
10. Observe whether the server independently verifies ownership before returning the object.
11. Record only the minimum response fields necessary to prove the authorization decision.

NST-001 CONTINUED

Broken object-level authorization - evidence and remediation

Redacted evidence

```
GET /v1/accounts/acct_demo_001 HTTP/1.1
Host: api.example.test
Authorization: Bearer SAMPLE-TOKEN

HTTP/1.1 200 OK
{"account_id":"acct_demo_001","owner":"user_test_a","plan":"demo"}
```

```
GET /v1/accounts/acct_demo_002 HTTP/1.1
Host: api.example.test
Authorization: Bearer SAMPLE-TOKEN # session for user_test_a

HTTP/1.1 200 OK
{"account_id":"acct_demo_002","owner":"user_test_b","plan":"REDACTED"}
```

WHY THIS MATTERS The validation changes only the object reference. A secure implementation should reject cross-owner access regardless of whether the identifier is guessable, random or obtained elsewhere.

Root cause

Object authorization is implemented inconsistently at the endpoint or data-access layer instead of being enforced through a centralized server-side policy that evaluates requester, resource, tenant and action together.

Remediation guidance

- Enforce object-level authorization server-side on every read, create, update and delete operation.
- Centralize authorization checks in a reusable policy or service rather than relying on client-provided identifiers or scattered endpoint conditions.
- Resolve resources through an authorization-aware query boundary, for example by scoping lookups to the authenticated principal or tenant before returning the object.
- Deny by default when ownership or permission context is missing or ambiguous.
- Add negative authorization tests covering cross-user, cross-tenant and role-boundary cases for related resource families.

Verification / retest criteria

Retest with two controlled accounts. user_test_a must retain legitimate access to acct_demo_001 while attempts to read or modify user_test_b resources are denied consistently. Equivalent negative tests should cover adjacent account-resource endpoints and relevant actions.

References: [CWE-639](#) | [OWASP API Security Top 10 - API1](#)

DETAILED TECHNICAL FINDING

7. NST-002 - Unsafe mass assignment in profile update

Severity	Medium
Status	Open
Affected component	api.example.test - profile update
CWE	CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attributes
OWASP mapping	OWASP API Security Top 10 - API3: Broken Object Property Level Authorization

Description

The profile-update workflow accepted an unexpected sensitive property that was not part of the intended user-editable profile. The behavior is consistent with directly binding client input to an internal data model without an explicit allowlist of fields permitted for the current user and operation.

Preconditions

A valid authenticated standard-user session and access to the in-scope profile-update workflow.

Technical risk

Over-broad server-side binding can let untrusted input modify internal attributes outside the intended update contract.

Sanitized validation procedure

12. Authenticate as user_test_a.
13. Submit a normal profile update containing only documented editable fields.
14. Repeat the request with one additional fictional sensitive property that should be server-controlled.
15. Verify whether the server ignores or rejects the unexpected field, or incorrectly persists it.

Redacted evidence

```
PATCH /v1/profile HTTP/1.1
Host: api.example.test
Authorization: Bearer SAMPLE-TOKEN
Content-Type: application/json

{"display_name":"Sample User","notification_opt_in":true,"account_tier":"REDACTED"}

HTTP/1.1 200 OK
{"display_name":"Sample User","account_tier":"REDACTED"}
```

Business impact

In a real system, unsafe binding can allow users to change sensitive properties that were intended to be controlled only by trusted server-side logic. The practical impact depends on which fields are exposed; privilege, workflow state, tenant linkage, entitlement or verification attributes are typical areas of concern.

Root cause and remediation

The root cause is overly broad model binding. Define operation-specific request schemas and explicit field allowlists. Treat sensitive attributes as server-controlled, enforce privilege and workflow rules independently of client input, and reject or ignore unexpected properties consistently. Do not rely on front-end visibility to protect server-side fields.

Retest criteria

Unexpected sensitive properties must be rejected or safely ignored, legitimate profile fields must continue to function, and role or entitlement changes must require the intended privileged server-side workflow.

References: [CWE-915](#) | [OWASP API Security Top 10 - API3](#)

DETAILED TECHNICAL FINDING

8. NST-003 - Insufficient rate limiting on a sensitive workflow

Severity	Medium
Status	Open
Affected component	api.example.test - recovery verification
CWE	CWE-799: Improper Control of Interaction Frequency
OWASP mapping	OWASP API Security Top 10 - API4: Unrestricted Resource Consumption (related abuse-control concern)

Description

A fictional recovery-verification workflow accepted repeated requests from the same controlled identity without meaningful progressive delay, cooldown or equivalent abuse control. This is a security interaction-frequency issue, not a general performance or scaling finding.

Preconditions

Access to the public or authenticated recovery-verification workflow and a controlled fictional account.

Technical risk

Repeated attempts can remain economically cheap for an attacker when attempt state is not constrained across the relevant identity and workflow context.

Sanitized validation procedure

16. Initiate recovery for a controlled fictional account.
17. Submit a small, bounded series of invalid verification attempts within the approved window.
18. Observe response status, delay, cooldown and security-event generation; stop well before disruptive volume.

Redacted evidence

```
POST /v1/recovery/verify HTTP/1.1
Host: api.example.test
Content-Type: application/json
```

```
{"account":"user_test_a","verification_code":"REDACTED"}
```

Bounded sample: consistent timing; no progressive delay or workflow cooldown.

Business impact

Insufficient abuse controls can make repeated guessing or workflow manipulation more practical. Impact depends on code strength, expiry, monitoring and whether attempts can be distributed across identities or sources.

Root cause

The workflow lacks a dedicated abuse-control policy that binds attempt state to the protected action and relevant identity context.

Remediation guidance

- Use layered limits across identity, account, endpoint and relevant behavioural or source signals, with progressive backoff or temporary cooldowns.
- Prevent trivial counter resets across new sessions or source changes, while avoiding controls that can be abused to lock out legitimate users.
- Log throttling and repeated failures with enough context for detection, alerting and investigation.

Verification / retest criteria

A bounded sequence of invalid attempts should trigger the designed abuse response, security telemetry should record the event, and legitimate recovery should remain usable after the documented cooldown or reset condition.

References: [CWE-799](#) | [OWASP API Security Top 10](#)

DETAILED TECHNICAL FINDING

9. NST-004 - Incomplete session-cookie security attributes

Severity	Low
Status	Open
Affected component	app.example.test - session cookie
CWE	CWE-614 / CWE-1004
OWASP mapping	OWASP Web Security Testing Guide - session management testing

Description

The fictional application issued a session cookie with incomplete browser security attributes. These flags affect when the browser sends the cookie and whether client-side scripts can access it.

Preconditions

A browser session established through an in-scope authenticated workflow.

Technical risk

Missing browser-enforced cookie restrictions can increase session exposure when combined with another client-side or cross-site weakness.

Attribute expectations

Attribute	Purpose	Sample expectation
Secure	Restricts cookie transmission to encrypted HTTPS connections.	Enabled for session cookies.
HttpOnly	Prevents JavaScript from reading the cookie through standard browser APIs, reducing exposure in some script-injection scenarios.	Enabled unless a documented design requires script access.
SameSite	Constrains cross-site cookie sending and can reduce some cross-site request risks.	Set deliberately to Lax or Strict where compatible; use None only when required and with Secure.

Applicable risk

This does not by itself demonstrate session compromise. Missing flags remove browser-enforced safeguards and can increase the impact of another weakness. Low severity is appropriate for this isolated configuration issue.

Sanitized evidence

Set-Cookie: session=REDACTED; Path=/; SameSite=Lax Observed in sample: Secure present, HttpOnly absent on one session-cookie variant.
--

Root cause and remediation

Cookie attributes are configured inconsistently. Define one shared session-cookie policy, enable Secure and HttpOnly for authenticated sessions, choose SameSite deliberately for real cross-site requirements, and add configuration regression checks.

Retest criteria

Authenticate through each in-scope session-establishment path and confirm all session-cookie variants include the expected attributes without breaking legitimate sign-in, sign-out or required cross-site behavior.

References: [CWE-614](#) | [CWE-1004](#) | [OWASP WSTG - Session Management](#)

POSITIVE SECURITY OBSERVATIONS

10. NST-005 - Controls that behaved as expected

Severity	Informational
Status	Observed
Affected component	Selected application and API (application programming interface) controls
Purpose	Acknowledge effective controls observed during the fictional assessment

Security reports should capture controls that worked correctly as well as weaknesses. The following fictional observations provide context and help engineering teams preserve useful security behavior during remediation.

Observed controls

Control	Observation	Why it helps
Transport security	All tested in-scope web routes consistently used TLS (Transport Layer Security), with no plaintext application workflow observed.	Protects data in transit against passive network exposure and supports secure-cookie enforcement.
Authentication responses	Selected sign-in failures used generic error messaging rather than revealing whether a specific test account existed.	Reduces straightforward account-enumeration signals.
Client-side storage	No session tokens or obvious sensitive profile data were observed in browser local storage during the sampled workflows.	Reduces persistence of sensitive data in script-accessible storage.
Input rejection	Selected server-side validation paths rejected malformed or unsupported values rather than relying only on browser validation.	Shows that some validation controls are enforced at the trust boundary.

IMPORTANT CONTEXT Positive observations apply only to the specific workflows and conditions tested. They do not prove that the same control is present everywhere and do not guarantee complete security.

Preservation during remediation

Engineering teams should keep these controls intact while fixing the confirmed issues. Regression tests should cover both the vulnerability fix and known-good behaviors so that remediation does not inadvertently weaken unrelated security controls.

REMEDIATION AND VERIFICATION

11. Remediation roadmap and retest process

Remediation roadmap

Horizon	Recommended actions	Outcome
Immediate	Correct object-level authorization; review related resource families; add negative authorization tests.	Restore tenant and object ownership boundaries.
Short term	Introduce field allowlists; strengthen abuse controls; improve security logging around sensitive workflows.	Reduce unintended state changes and make repeated abuse more detectable and costly.
Ongoing	Integrate authorization regression testing; review sensitive workflows after major changes; schedule periodic security assessments.	Build repeatable assurance into the development lifecycle.

Retest process

For a focused retest, the client should provide a brief remediation note for each finding, deployment confirmation for the test environment, any changed endpoint or workflow details, and controlled accounts or permissions needed to reproduce the original condition safely.

Status definitions

Status	Meaning
Resolved	The original condition is no longer reproducible and the implemented control addresses the reported root cause for the agreed test cases.
Partially resolved	Risk is reduced, but one or more original conditions, equivalent paths or required controls remain incomplete.
Not resolved	The original condition remains reproducible or the supplied change does not materially address the reported issue.

A retest is not a complete second penetration test. It is a targeted verification of agreed findings and related bypass checks within the retest scope. New functionality, unrelated attack surface and broad regression coverage require separate assessment scope.

Updated reporting

The final report can retain the original finding text and add a clearly dated retest note showing the updated status, verification evidence summary and any residual risk. This preserves auditability without erasing the original assessment record.

CLOSE-OUT

12. Conclusion

This fictional assessment illustrates a practical reporting approach: prioritize material risk, explain why the weakness matters, give developers enough context to fix the root cause, and define objective retest criteria. In the Northstar SaaS Demo scenario, object-level authorization is the first remediation priority, followed by safer field handling, stronger workflow abuse controls and standardized session hardening.

Penetration testing remains a point-in-time activity. Durable security comes from continued secure-development practices, explicit authorization design, regression testing, useful security telemetry, disciplined change review and periodic independent assessment.

SAMPLE STATUS Sanitized fictional sample - not a client engagement. No real client, employer, production system, endpoint, credential or customer data appears in this report.

Request a Security Assessment

For an authorized assessment, scope and testing conditions should be agreed in writing before testing begins. A typical engagement can include authenticated web and API testing, authorization and business-logic review, cloud-facing attack-surface validation, remediation guidance and a focused retest.

Vinit Vora	Application Security Engineer
Specialties	Web & API Penetration Testing
Additional focus	AWS & Kubernetes Security
Email	contact@vinitvora.com
Website	https://vinitvora.com

vinitvora.com | contact@vinitvora.com